RATAN TATA MAHARASHTRA STATE SKILLS UNIVERSITY

School of Science Engineering and Technology Skills

Department of Engineering

Subject – DevOps and Automation Lab

Lab Assignment 2

Aim:

Create a local Git repository and perform basic operations

(add, commit, status, history, branching)

Learning Objectives-

After completing this lab, the student will be able to:

1. Understand the concept of version control and the role of Git in software development.
2. Create and initialize a local Git repository.
3. Track file changes using Git commands.
4. Perform basic Git operations such as add, commit, push, and pull.
5. Connect a local repository to a remote repository (GitHub).
6. Apply Git for collaborative and version-controlled development.

Theory

Version Control System (VCS)-

- **Definition and need:** A Version Control System (VCS) is a software tool that tracks and manages changes made to files over time. It enables developers to

maintain multiple versions of a project, revert to previous states when required, and collaborate efficiently without overwriting each other's work.

- **Problems solved by VCS in software projects**

    1. Prevents loss of previous file versions
    2. Enables collaboration among multiple developers
    3. Tracks who made changes and when
    4. Allows rollback to stable versions in case of errors
    5. Maintains organized project history

Git-

- **Definition:** Git is an open-source, Distributed Version Control System (DVCS) created by Linus Torvalds in 2005. It is designed to handle everything from small to very large projects with speed and efficiency

- **Features of Git:**

    1. **Distributed Nature:** Every user has a complete copy of the repository
    2. **High Speed:** Fast operations for commits and branching
    3. **Branching and Merging:** Supports parallel development
    4. **Data Integrity:** Uses cryptographic hashing to secure data
    5. **Offline Access:** Work can continue without internet connectivity

- **Difference between Git and centralized VCS**

    In a CVCS, there is a single central server; if it goes down, no one can collaborate or save history. In Git, every local clone acts as a full backup, providing higher redundancy and offline capabilities.

Aspect	Git (Distributed VCS)	Centralized VCS
Repository storage	Each user has a full copy	Single central server
Offline work	Possible	Not possible
Speed	Faster operations	Slower due to server dependency
Collaboration	Peer-to-peer sharing	Centralized control

Local Repository-

- **Meaning:** A local repository is a Git repository stored on a developer's computer containing project files and their complete version history.
- **Purpose of initializing a repository:** Initializing a repository creates a .git directory that enables Git to track file changes, manage commits, and maintain project history.

Remote Repository-

- **Definition:** A remote repository is a Git repository hosted on an online server that allows multiple users to access, share, and collaborate on a project.
- **Role of GitHub in Git-based projects:** GitHub is a popular hosting platform for remote Git repositories. It provides tools for collaboration, code review, issue tracking, and project management.

Git Working Areas-

1. **Working Directory:** The actual folder on your disk where you create, edit, and delete files. These changes are "untracked" or "modified" until staged.
2. **Staging Area (Index):** A virtual "holding zone" for changes. It allows you to select exactly which modifications will be included in the next commit.
3. **Local Repository:** The permanent storage area where Git saves the snapshots (commits) of your project

Basic Git Commands-

- **Initializing repo (git init):** Creates a new .git subdirectory in your current folder, turning it into a tracked repository.
- **Staging (git add):** Moves changes from the working directory to the staging area. It signals to Git that these changes are ready to be saved.
- **Commit (git commit):** Captures a snapshot of the staged changes and records them permanently in the local repository's history.
- **Checking Status (git status):** Displays the state of the working directory and staging area, showing which files are modified, untracked, or staged. ·
- **Branching (git branch):** Creates a new pointer to the current commit, allowing you to develop a "feature" in a separate timeline without affecting the main code.
- **Switching Context (git checkout/git switch):** Navigates between different branches by updating the files in your working directory to match the version stored in the target branch.

Commit-

- **Meaning:** A commit is a snapshot of the project at a specific point in time, permanently recorded in the repository history.
- **Importance of commit messages:** Commit messages describe the changes made in a commit. Clear messages help in tracking progress, understanding project evolution, and simplifying debugging.

Execution Procedure:

1. **Initializing repo: git init**

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2
$ git init
Initialized empty Git repository in C:/Users/shrey/Documents/MSSU/MSSU_Sem_6/Dev
Ops/DevOpsLab2/.git/
```

2. Checking status of files: git status

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2 (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        demo_git.html

nothing added to commit but untracked files present (use "git add" to track)
```

3. Staging: git add

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2 (master)
$ git add -A
warning: in the working copy of 'demo_git.html', CRLF will be replaced by LF the
 next time Git touches it
```

4. Commit: git commit

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2 (master)
$ git commit -m "Initial Commit"
[master (root-commit) 06649b2] Initial Commit
 1 file changed, 12 insertions(+)
 create mode 100644 demo_git.html
```

5. Working with a feature branch: git branch feature_branch

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2 (master)
$ git branch feature_branch
```

6. Switching Context: git checkout / git switch

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2 (master)
$ git checkout feature_branch
Switched to branch 'feature_branch'

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/DevOpsLab2 (feature_bran
ch)
$
```

Questions:

1) What is the purpose of the git init command?

The git init command is used to **create a new Git repository in a project directory**. When this command is executed, Git generates a hidden folder named .git, which contains all the necessary metadata and configuration files required to track changes in the project. This marks the directory as a Git repository and allows the user to begin version controlling the project. After initialization, files in the folder can be added, committed, and managed using Git commands. Essentially, git init is the first step in setting up version control for any new project.

2) Differentiate between a local repository and a remote repository.

A **local repository** is a Git repository that resides on a developer's personal computer. It contains the project files along with the complete commit history. Developers perform operations such as editing files, staging changes, and committing updates within the local repository. Since it exists on the local machine, it allows work to continue even without an internet connection.

A **remote repository**, on the other hand, is a Git repository hosted on a server or cloud-based platform such as GitHub, GitLab, or Bitbucket. Its main purpose is to enable collaboration by providing a shared space where multiple developers can push their local changes and pull updates made by others. Remote repositories also serve as backup storage and help in project distribution and team-based development.

In summary, the local repository is for individual development and offline work, while the remote repository is for sharing, collaboration, and centralized project access.

Aspect	Local Repository	Remote Repository
Storage Location	Resides on the developer's local system	Hosted on a remote server or cloud-based platform
Core Function	Facilitates local development and version tracking	Enables code sharing and collaborative development
Network Dependency	Operates independently without internet access	Requires internet connectivity for synchronization
Repository Content	Maintains complete project files and commit history	Holds a centralized copy of the project repository
Examples	Project directory containing a .git folder	GitHub, GitLab, Bitbucket repositories

3) What does the git status command display?

The git status command displays the **current state** of the **working directory** and the **staging area.** It informs the user which files have been modified, which files are staged and ready to be committed, and which files are untracked by Git. Additionally, it indicates whether the local branch is ahead of or behind the remote branch. This command is extremely useful for developers to understand the present condition of their repository before performing operations like adding or committing files.

4) What is the function of the git add command?

The git add command is used to move changes from the **working directory** to the **staging area**. When files are modified, Git does not automatically include them in the next commit. The git add command explicitly tells Git which files or changes should be included in the upcoming commit. It allows developers to selectively stage specific files, giving better control over what gets recorded in each commit. Without using git add, changes made in files will not be saved in the repository history.

5) Why are commit messages important?

Commit messages are **short descriptive texts** written when creating a commit. They **explain what changes** were made in that particular commit and **why those changes** were necessary. Well-written commit messages help developers and team members understand the evolution of the project over time. They make it easier to locate specific updates, identify when bugs were introduced, and review project progress. In collaborative environments, commit messages serve as a communication tool among developers, improving clarity and maintainability of the codebase.